

Vrije Universiteit Amsterdam



Bachelor Thesis

The Three-Dimensional Open Dimension Problem: Synthesizing Strategies for Efficient Bin Packing

Author: William Mooijer 2594900

Supervisor: Daan van den Berg
Second Reader: Bob Borsboom

*A thesis submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science in Artificial Intelligence*

June 18, 2026

*“As a child, I considered such unknowns sinister.
Now, though, I understand they bear no ill will.
The universe is, and we are.”
Solanum, Outer Wilds*

Abstract

This thesis explores a variant of the 3D bin packing problem whereby orders of items are to be packed in a single bin as efficiently as possible, minimizing the height of the packed mass of items, called the three-dimensional rectangular open dimension problem. Using real-world orders and box dimensions from the Centraal Boekhuis company, as well as a custom-built pallet environment simulation and visualization suite, a set of three classical algorithms was implemented to test their performance against a bespoke branch and bound algorithm. This algorithm synthesizes established techniques like extreme point generation and symmetry breaking, as well as new techniques like deduplication and top-X limiting for candidate selection, search tree optimization, and branch filtering. Notably, the top-X branching filter introduced in this work as a variation of beam search designed to address its inherent weaknesses while leveraging its strengths, allowed the branch and bound algorithm to complete all Centraal Boekhuis orders in tractable computation times. Computational experiments on a provided order set of 40 orders and a generated set of 3000 testing orders showed that the new algorithm outperformed the best fit decreasing algorithm in every class of problem difficulty. While the exponential complexity of the problem remains a limitation for the largest orders, the proposed top-X branching filter still presents a promising improvement over beam search methods, offering a viable path for real-world logistical optimization.

Contents

1	Introduction	1
2	Background and Related Work	2
2.1	Problem Typology	2
2.2	Greedy Algorithms	2
2.3	Placement Candidates	3
2.4	Branch and Bound	3
2.5	Exact Methods	4
2.6	Beam Search	4
2.7	Symmetry breaking	5
3	System and Environment Modeling	5
3.1	The Box and Order Model	5
3.2	Placement Rules and Validation	6
3.3	Objective Function and Evaluation Metrics	7
4	Algorithms	8
4.1	Random	8
4.2	First Fit Decreasing	8
4.3	Best Fit Decreasing	10
4.4	Branch and Bound	10
4.4.1	Overview	10
4.4.2	Initialization and Warm-start	11
4.4.3	Recursive Structure	11
4.4.4	Bounding Rules	11
4.4.5	Branching Filters	12
5	Experiments and Results	14
5.1	Random Algorithm Order Fulfillment Test	14
5.2	Branch and Bound Development	15
5.3	Optimality Guarantee Test	17
5.4	Top-X Limit Test	18
5.5	Algorithm Comparison	19
6	Discussion	20
6.1	FFD Inapplicability	20
6.2	Top-X Limiting Weakness	20
6.3	Physics Limitations	21
6.4	Branch and Bound Failures	21
7	Conclusion	21
	References	23
A	Appendix	25
A.1	Source Code	25
A.2	Reproducibility	25
A.3	Full Results Figures	25

1. Introduction

The Centraal Boekhuis company is the largest logistical service provider of Dutch-language books in digital and physical formats, forming the backbone of book distribution between publishers, sellers, resellers, and consumers of various scales. Because the company has been in the business for more than 150 years, a lot of work has gone into its packing process, as every cubic centimeter per pallet saved can add up to significant monetary and environmental benefits.

In this work, both classic and modern computational methods are used to attempt to find the most effective way of packing orders of up to 40 boxes onto a fixed pallet space, in a way whereby the maximum height of the total packed volume is minimized. The types and sizes of boxes, as well as the makeup and size of the 40 different box sets for packing that are used in this work are all taken from the real-world data of Centraal Boekhuis, giving this work the potential to make a meaningful difference in optimizing actual physical logistics matters.

The problem of trying to find an optimally efficient packing for three-dimensional shapes in three-dimensional spaces is called the three-dimensional bin packing problem (*3D-BPP*). The 3D-BPP, along with the related Knapsack Problem, was found to be among the top five most popular NP-hard problems by Christensen *et al.* (2016) (2). A potential reason for that is that the 3D-BPP is a problem that is simple to convey but extraordinarily complex in its execution: the solution space of different box placements exhibits severe combinatorial explosion, growing exponentially as the number of boxes to be packed increases.

The problem as presented in this work concerns the orthogonal packing of strictly cuboid-shaped items (*boxes*) of 10 classes (*box types*) consisting of 9 different length, width, and height combinations (*box dimensions*), with box dimensions ranging from 30 to 455 millimeters on the different axes, in a bounded space (*the pallet*) of a fixed 1000 by 1400 by 1400 (length, width, and height, respectively) millimeter size. The 40 sets of boxes to be fitted on the pallet (*orders*) from the Centraal Boekhuis data (*Centraal Boekhuis orders*) range in number from 30 to 40 boxes, and are numbered 1 through 40.

In the field of computer science, a problem is classified as NP-hard if it is as difficult to solve as the hardest problems in the NP-class, which is a class of problems for which no general polynomial-time algorithm capable of solving them exists. In other words, if the solution to a problem can be checked easily, but cannot be synthesized easily, it is classified as an NP-complete problem, placing its computational difficulty in the NP-hard category. A subset of NP-hard problems is classified as “strongly NP-hard”. A problem is classified as strongly NP-hard if it retains NP-hardness even with low input parameters. In the case of the 3D-BPP, that would mean only having to place a small number of boxes. As a consequence, a strongly NP-hard problem cannot be solved by pseudo-polynomial time algorithms (Garey & Johnson, 1979) (6). The problem’s hardness to solve thus does not stem from the magnitude of its inputs, but rather stemming from its fundamental combinatorial structure.

The 3D-BPP is a well-known example of this extreme computational difficulty. Formally, the 3D-BPP is considered strongly NP-hard. This is proven by the fact that the one-dimensional version of this problem, the 1D-BPP, is also proven to be strongly NP-hard by showing that it is a reduction of the 3-partition problem, a foundational strongly NP-hard problem (Coffman et al., 1984 (3); Garey & Johnson, 1979 (6)). Because the 1D-BPP is a special case of the 3D-BPP such that all but one dimension of every item to pack is reduced to 1 or some other constant, the 3D-BPP retains the strong NP-hardness of the 1D-BPP (Zhao *et al.*, 2021) (16).

2. Background and Related Work

The three-dimensional bin packing problem is a very heavily researched problem. With more than 4,000 published papers covering it in the year 2025 alone, and more than 37,000 in total, this specific problem has received more scientific attention than some whole scientific fields.

2.1 Problem Typology

The problem addressed in this work and its related variants, collectively called cutting and packing problems, have reached such levels of research saturation that even in 1990, the need to firmly differentiate and categorize them was identified by Dyckhoff (1990)(5).

Dyckhoff identified nine distinct characteristics of this problem, such as quantity measurement (discrete or continuous) and the geometric representations (regular or irregular) of the objects to cut or pack. Based on those, a 4-character typology was established based on four primary criteria of differentiation, with each character denoting the problem’s type under one criterion. These criteria are dimensionality (the difference between one-dimensional problems like computer memory allocation (1), two-dimensional problems like glass sheet cutting (2), three-dimensional problems like bin packing (3), and higher-dimensional problems like bin packing with per-box temporal constraints (4 or more)), the kind of assignment (the difference between having to fit every box on an order list on as few pallets as possible (V) and fitting as many boxes as you can on a single pallet (B)), the assortment of large objects (the difference between packing a single pallet (O), packing multiple identical containers (I), and packing products into differently-sized shipping boxes (D)), and the assortment of small items (the difference between packing identical items (C), packing many items of a few types (R), packing many items of many different types (M), and packing few items of unique items (F)). This framework provided a much-needed shorthand for problem types. The Dyckhoff classification of the problem addressed in this thesis is 3/V/D/R, as it concerns a three-dimensional packing problem where every order must be fully packed on pallets that can have differing height measurements to fit, and every order can only consist of at most ten different types of boxes.

Wäscher *et al.* (13) improved upon the typology established by Dyckhoff by simplifying sequence criteria and adding a criterion for the shape of small items in order to distinguish between regular and irregular shapes of items. Materially, this typology fixed the tendency of Dyckhoff’s typology to identically classify fundamentally different problems. This can be illustrated with the difference between the classification of the classic 3D-BPP and the problem addressed in this work. Dyckhoff’s system classifies the former as 3/V/I/M, and the latter as 3/V/D/R. The difference in assortment of small items (M vs. R) is a consequence of this work dealing with box types rather than unique boxes, but the difference in assortment of large objects (I vs. D) shows a weakness in Dyckhoff’s system. Even though the bins in this work have a fixed size, the optimization for a single dimension surpasses proper classification, and must thus be categorized as a problem whereby a choice of bin must be made from a supply of bins of every possible height measurement, giving the D classification. This weakness was resolved by Wäscher *et al.* with the introduction of variable dimensions, allowing the proper classification of the problem discussed in this work as a three-dimensional rectangular open dimension problem (*3D-RODP*).

2.2 Greedy Algorithms

In NP-hard problems, heuristic solving methods are often sought out to provide acceptable solutions in feasible timeframes. This work is no different. In Chapter 4, the baseline heuristic algorithms used as baseline performance comparisons, called first fit decreasing (*FFD*) and best fit decreasing (*BFD*), are introduced. These algorithms originated in a foundational doctoral

dissertation by David S. Johnson (1973) (7) on the 1D-BPP. They follow the same two-phase structure, in which the items to be packed are first sorted in decreasing order of size, then placed sequentially based on selection rules. It is worth noting that this dissertation concerns the classic version of the packing problem, which is about fitting the items to pack in as few bins as possible, but most of the concepts are common between problems in this category.

The FFD algorithm simply places every item it encounters into the first bin that will fit. BFD, however, improves upon this method by placing each item into the bin that minimizes the space left in the bin. In the case of a tie in this calculation, the item is packed into the first of the tied bins. Both of these algorithms are considered greedy constructive heuristics: while placing boxes, they commit to their placements irrevocably, and there is no backtracking. The key factor that separates FFD and BFD from other similarly non-backtracking algorithms is the initial sorting phase. Starting with the largest item decreases spatial fragmentation in the bins, leaving more flexible volumes of space that can be used by the smaller items, as noted by Zanchetta (2025) (14).

In this work, FFD and BFD serve as the primary baselines that the proposed branch and bound algorithm will be compared against. In addition, BFD is used as a “warm-start” method (see Section 2.4) in the proposed algorithm, providing a good starting upper bound to initialize the search.

2.3 Placement Candidates

One of the most fundamental challenges in any bin packing problem is the fact that, being continuous, spaces of any dimensionality and size have an uncountably infinite amount of possible placement points. As a consequence, any naive item placement algorithm will inevitably end up being unable to provide a solution in feasible computation time. Early attempts to tackle this challenge included the discretization of these continuous spaces. As a consequence, however, this greatly exacerbated the issues of computational complexity, memory requirements, and processing speeds (Fernandez, 2018) (8).

To address this issue of complexity, Martello *et al.* (2000) (11) introduced the ‘corner points’ heuristic (see Figure 2.1a). This heuristic restricts placement candidates to points where the envelope of packed items changes slope, based on the proven principle that optimality in packing is preserved while pushing items as far downward, leftward, and backward as possible. As an improvement to this technique, Crainic *et al.* (2008) (4) developed the ‘extreme points’ algorithm (see Figure 2.1b), decoupling placement candidate selection from the overall packing geometry. Upon placement, item edges are projected toward the bin origin until they intersect with the boundaries of another item or the bin, placing an extreme point where the intersection occurs. This losslessly compresses an infinite set of placement coordinates into a small set of discrete candidates. A version of this algorithm is also used in this work (see Section 3.2).

2.4 Branch and Bound

Even though it is a very old technique in programming, originally designed as a method for solving discrete programming problems and proposed over sixty-five years ago by Land & Doig (1960) (9), the *branch and bound* methodology has seen extensive use and enabled much success in the field of packing problems, and computer science as a whole. In the context of packing problems, Martello *et al.* (2000) (11) formally introduced a landmark branch and bound algorithm that preserved the certainty that the algorithm will find the optimal solution (the *guarantee of optimality*).

Branch and bound algorithms work by recursively exploring the space of possible solutions from every partial solution, creating branches in a search tree (branching) and pruning those branches that are known not to lead to a favorable outcome (bounding). In their simplest forms

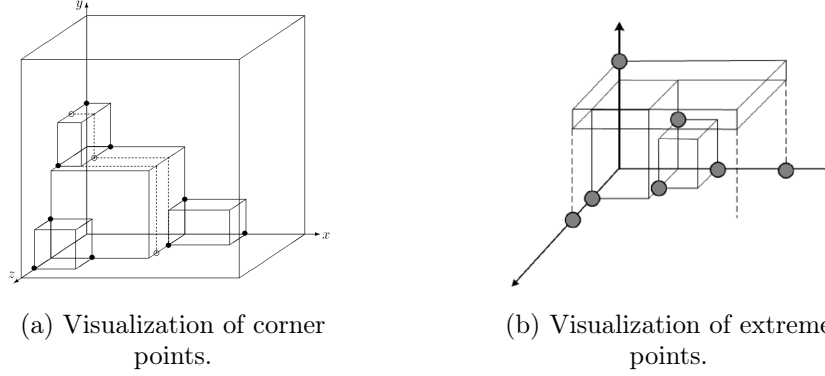


Figure 2.1: Comparison of placement candidates generated by Martello *et al.*'s corner points (11) (a) and Crainic *et al.*'s extreme points (4) (b)

in the context of the problem addressed in this work, branching can be thought of as ‘considering’ placing a box on any of the available placement candidate positions, and bounding can be thought of as assessing a certain state of the bin, concluding that the remaining boxes cannot fit in the bin’s current arrangement, and abandoning the consideration of this state and all its potential future states.

As can be surmised from the conceptual explanation, a branch and bound algorithm is only as good as its capacity to assess and prune bad branches. The default bounding rule that 3D-BPP branch and bound algorithms share is that a partial state that is assessed as already scoring worse than the best known completed state is immediately pruned. The branch and bound algorithm proposed in this work uses this as its basis. A technique to sharpen this best known score before even initiating the tree search is to run a simpler algorithm like BFD (see Section 2.2) and use its solution as a known upper bound for the branch and bound algorithm to compare against. This is known as “warm-starting”.

2.5 Exact Methods

Building on the knowledge discussed above, Martello *et al.* (2000) (11) also introduced a foundational two-level branch and bound algorithm to find optimal solutions for the 3D-BPP. An important part of their success was the establishment of accurate lower bounds, from the absolute lower bound (in which shapeless volumes and perfect packing are assumed), called L_0 , which is shown to underestimate the actual optimal packing by up to a factor of eight, to the much more accurate L_2 , derived by reduction to one-dimensional bin packing, which achieved an average deviation of 9.6% from optimal, compared to 28.4% for L_0 . Through the use of these highly accurate bounds, as well as a two-level decomposition in which a main decision tree is used to assign items to bins, as well as a sub-algorithm that verifies the item’s fit using corner points, optimal solutions to the 3D-BPP could be found in reasonable computation times.

2.6 Beam Search

Beam search is an adaptation for branch and bound algorithms that aims to counter catastrophic combinatorial explosion in search spaces by strictly limiting the size of the search tree. In a doctoral thesis by Lowerre (1976) (10), the technique is introduced as a critical enabler for a continuous human speech recognition system.

Rather than a best-first search with backtracking, which critically lacked in computational speed, or a breadth-first search, which led to an exponential explosion of memory requirement, beam search was developed as a way to combine the strengths of tree search with a rigid maximum to limit computational complexity. Shown in the application by Araya *et al.* (2020) (1), beam

search evaluates every potential child node from each node at the current search tree depth, ranks them, and only populates the next level down with the top k evaluated nodes. This way, the search tree keeps a constant width at every node, keeping the shape of a straight beam rather than a branching tree. In this work, a variation of this technique is introduced called Top-X Limiting (Section 4.4.5).

2.7 Symmetry breaking

In the context of branching algorithms for complex problems, it is all but assured that there exist situations whereby two or more different branches represent or lead to an identical state. These are called symmetrical states. In the context of the 3D-BPP, this can frequently happen when multiple items with identical dimensions are packed. A naive implementation of a branch and bound algorithm would treat each placement as a completely unique state, even when, by every identifiable metric, the state of the bin is identical. As a consequence, this exacerbates the already significant problem of combinatorial explosion, without the benefit of exploring new bin states.

Finding a good way of dealing with symmetrical states is a critical component of cutting down on inefficient computation in branching algorithms. In their thesis on vehicle routing problems, Zhang (2021) (15) reduced redundant computation by evaluating whether states are geometrically covered or dominated by other states. If a state is found to be covered or dominated by another, it is safely pruned. Rules like these are called dominance rules. Based on these methods of streamlining branching computation, this work also incorporates a method of breaking symmetry (see Section 4.4.5).

3. System and Environment Modeling

In the virtual model of the pallet, the x -dimension, y -dimension, and z -dimension correspond to length, width, and height, respectively. In this work, the primary unit of measurement is the millimeter (mm). To illustrate: the physical space of the pallet can be denoted by three coordinates (x, y, z) , which range from $(0, 0, 0)$ corresponding to the left, back, bottom corner, to $(1000, 1400, 1400)$ corresponding to the right, front, top corner. The pallet is virtually represented by a 2-dimensional, 1000 by 1400 *NumPy* array in which each element at index $[x, y]$ indicates the height of box mass at coordinates (x, y) . This, along with a list of the sizes and locations in three-dimensional space of the boxes placed on the pallet, forms the entirety of the virtual pallet state from which all visualizations and metrics can be extracted. One of these visualizations can be seen in Figure 3.1

3.1 The Box and Order Model

In the given problem statement, ten types of cuboid-shaped boxes are defined, along with their dimensions in the x , y , and z dimensions. The chosen method for representing these values is as a `.csv` file (*boxtypes.csv*), in which each box type is given an ID from 1 to 10 for brevity, and stored in the order of *ID*, *Name*, *Length*, *Width*, and *Height*. For example, the encoding of the largest box by volume is:

3, *DOOSGR*, 455, 310, 200.

In the virtual representation, the ability to rotate the boxes in the horizontal or vertical planes can be allowed or disallowed separately. For all of the testing in this work, both types of rotations are allowed. The problem statement also provides a list of multisets of box types. These are referred to as *orders*, and are similarly encoded in a `.csv` file (*orders.csv*). They are assigned

an order ID and encoded in the format *order_id*, *amt_1*, *amt_2*, ..., *amt_10*, where *amt_X* signifies the amount of box ID *X* in that order. For example, the encoding of order number 16, which comprises 32 boxes, is:

16, 4, 8, 5, 0, 0, 0, 2, 4, 6, 3

The given problem statement contains 40 orders (which are referred to as *Centraal Boekhuis orders*), with the number of boxes per order ranging from 30 to 40, distributed to favor the higher end of that range, with a median box amount of 37.

To facilitate testing, another .csv file was created to contain orders useful for testing the code in different ways (*orders_test.csv*). Two types of these *test orders* were used:

41 type I test orders were created and assigned IDs 0 to 40, whereby the order ID is equal to the number of boxes in the order. The box quantities are constructed such that every type is equally represented, with the first ID *mod* 10 box types receiving one additional box. This list was used for benchmarking the speed of different box placing algorithms against a fixed order for each box amount.

Next, 3000 type II test orders were generated and assigned IDs 1000 to 3999 for clear distinguishing between the type, and to ensure that the first two digits of the ID indicate the number of boxes in that order. For each box count, 100 orders were generated with random distributions of the ten box types. These orders were used for the mass testing of the performance of different algorithms as well as different parameter sets (see Chapter 5).

The foundation for translating the separate data files into usable in-memory structures is the loading of the box types, Centraal Boekhuis orders, and test orders into separate Python *dict* objects that are kept in memory to enable constant-time lookup. In this transposition, the orders are transformed by turning them from multisets of box types into lists of box IDs to pack. For example, the list corresponding to the example order above (Centraal Boekhuis order 16) is as follows:

[1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 7, 7, 8, 8, 8, 8, 8, 9, 9, 9, 9, 9, 9, 10, 10, 10]

From here, any order can easily be interacted with in an iterable form. As the box ID is the primary method of identifying the properties of the object in question, a number of programmatic features can be utilized with a box list consisting of these IDs.

Firstly, box IDs can be used to access the physical dimensions, as well as their area and volume. Based on this, the list can be sorted based on any dimensional size criterion, as well as area or volume. Sorting the box list to pack by volume is the first step in each box placement algorithm, with the exception of the random placement algorithm.

Based on the fact that it is assumed that packages of rigid hardcover books are being packed, any orthogonal rotation of boxes is allowed for the purposes of this work. For any box, its set of valid dimensions (*dx*, *dy*, *dz*) (short for delta-x, delta-y, and delta-z, called a *dimension tuple*) can be generated in different orders, based on whether horizontal rotation and vertical rotation are allowed. For example, the valid *orientation tuples* for box type 1 are as follows:

[(62, 246, 349), (62, 349, 246), (246, 349, 62), (246, 62, 349), (349, 62, 246), (349, 246, 62)]

3.2 Placement Rules and Validation

To enable the virtual environment to accept box placements from algorithms, a number of pre-requisites are needed. An algorithm needs a set of candidate placements to choose from, and the environment itself needs a way of validating a box placement before executing it, and for more advanced algorithms, a way of undoing box placements.

To avoid evaluating every potential three-dimensional coordinate for each box placement, a method to narrow down candidate placements is required. In the early stages of this work, this

was achieved by keeping a record of candidate placements consisting only of existing box edges, thus guaranteeing that boxes are either placed in the $(0, 0, 0)$ coordinate or against another box, with the intention of leaving as few gaps as possible. This method of box placement candidate finding is used by the FFD and BFD algorithms (see Chapter 4). Later, this method was refined to an extreme points algorithm, similar to that of Crainic *et al.* (2008) (4). This is due to the fact that placing boxes anywhere other than on the intersection of two existing boundaries leads to small gaps that are unable to fit other boxes and are thus guaranteed to be wasted space. This adapted extreme points candidate selection method is used by the branch and bound algorithm presented in this work (see Section 4.4).

To place a box after candidate selection, all that is required to be transmitted are the x and y coordinates of the left, back, bottom corner of the box in the placement, as well as its dimension tuple. From here, the z -coordinate of the placement can be simply discerned by finding the maximum height of occupied space in the rectangle formed by the four points (x, y) , $(x + dx, y)$, $(x, y + dy)$, and $(x + dx, y + dy)$. For a box placement to be considered valid, the base of the box to be placed must be at least 70% supported by existing surfaces to approximate real-world physical stability. In addition, the dimensions of the box are checked against the pallet boundaries to see if they would be breached by placing this box, invalidating the placement if this would be the case.

If the placement is valid, the box can be placed. This involves raising the heightmap in the box area to its placement height plus its own height, and adding the box's x , y , z , dx , dy , and dz to the list of placed boxes. Apart from this, a copy is made of the box's dimensions and placement, the heightmap in the box area before and after placement, the extreme points added and removed by this placement, as well as the new edges introduced, and this copy is saved. This information package contains all the necessary data to completely reverse a box placement, and allows the graceful execution of this reversal by restoring the old data that was changed by a previous placement.

3.3 Objective Function and Evaluation Metrics

To measure the performance of any box placing algorithm, objective functions and/or evaluation metrics are required. The given problem statement dictates the primary objective: to keep the maximum z value of occupied space on the pallet as low as possible after fitting an order onto the pallet. However, that is not the only way to evaluate a pallet state. A number of other metrics were considered and created for this work.

It is important to know if all of the boxes in the relevant order were successfully placed on the pallet. This is counted as a percentage called *order fulfillment*. Second, the proportion of volume under the highest occupied level in the z -dimension (*z-height*) that is filled by boxes, rather than empty space, is considered and saved as *volume utilization*. This serves as a coarse-grained general overview of packing efficiency. To further refine this concept, a similar metric was conceived and named *packing score*, whereby the volume utilization of boxes is weighted based on z -height, with tighter packing at lower z -heights contributing the most to the final score. In addition, the height of the center of gravity of the placed box mass is calculated, assuming equally distributed constant density across all box types, and displayed as *CoG Z-height*. Finally, the percentage of the floor of the pallet occupied by boxes, *area filled at $z=0$* is calculated.

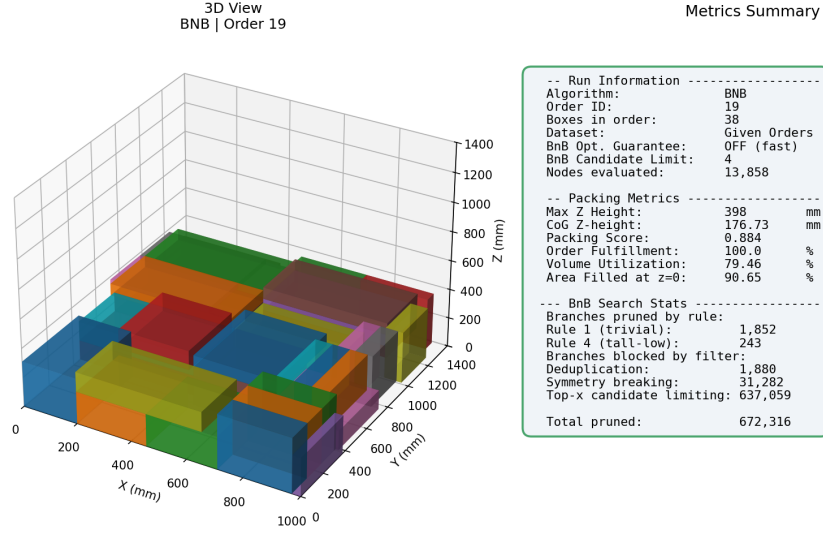


Figure 3.1: An example of a pallet visualization, including 3D box mass representation, order information, and calculated metrics.

4. Algorithms

4.1 Random

This algorithm, the simplest and first to be implemented, loops through all box IDs in the order to pack. For each box, it attempts to place it in a random (x, y) location on the pallet without changing its orientation, repeating the attempt a number of times configurable by setting the `max_attempts` value when calling the function. The default maximum number of attempts to make is 10,000. Until the box is placed successfully or the maximum number of attempts is reached, the algorithm will keep attempting random in-bounds locations to place the box. If the placement is unsuccessful in the allotted attempts, the box placement will simply be skipped.

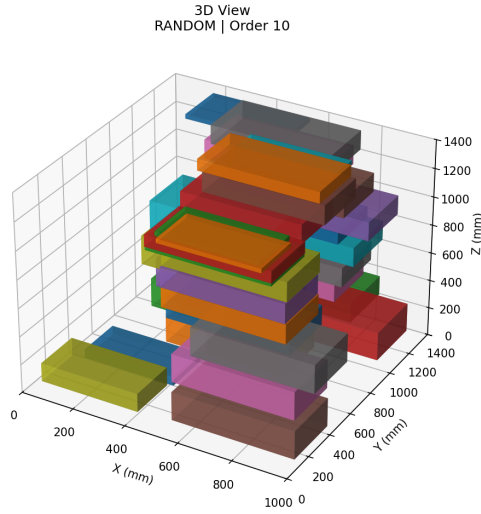
Because the chance that not all boxes in an order are successfully placed by this algorithm is significant, an experiment has been conducted to examine how different `max_attempts` values affect order fulfillment (see Section 5.1). Furthermore, every box placement attempt is counted as it is conducted for purposes of comparing the number of evaluated placements between algorithms. An example of an order placed by the random algorithm can be seen in Figure 4.1

4.2 First Fit Decreasing

For this work, the first fit decreasing algorithm introduced by Johnson (7) was adapted for use in the three-dimensional rectangular open dimension problem. Operating on a two-phase sort-and-place principle, the FFD algorithm is capable of placing items faster than any other algorithm discussed in this work (see Section 5.5).

As mentioned, the algorithm starts by sorting the boxes in the order to place. In this work, orders are sorted by volume in descending order unless specified otherwise. After this, the algorithm iterates over every box in the list. For every box, all orientation tuples are generated, and the pallet's candidate points are loaded. For each candidate point and orientation combination (with the order of priority being x -coordinate, y -coordinate, orientation), box placement attempts are made until the box is successfully placed, similarly to the random algorithm. Again, each placement attempt is counted as it is conducted for comparison purposes.

Pallet Results: **RANDOM** | Order 10 | Given Orders



Metrics Summary

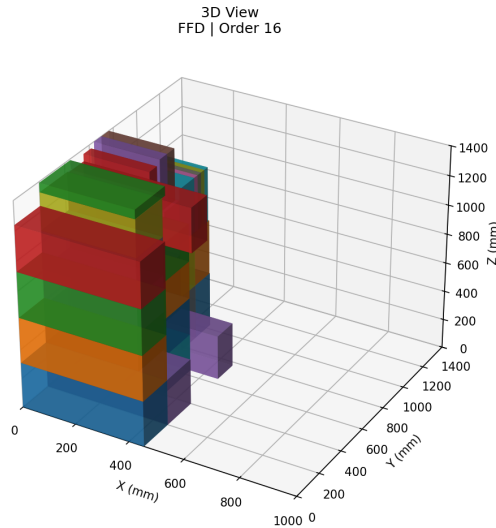
-- Run Information -----		
Algorithm:	RANDOM	
Order ID:	10	
Boxes in order:	38	
Dataset:	Given Orders	
BnB Opt. Guarantee:	N/A	
BnB Candidate Limit:	N/A	
Nodes evaluated:	11,484	

-- Packing Metrics -----		
Max Z Height:	1382	mm
CoG Z-height:	657.64	mm
Packing Score:	0.316	
Order Fulfillment:	86.84	%
Volume Utilization:	30.1	%
Area Filled at z=0:	43.32	%

Figure 4.1: An example of random algorithm box placement.

Because Johnson's (7) original FFD algorithm was created for the classic 3D-BPP, an adaptation had to be made for this work, both programmatically and in the meaning of 'first fit'. As the iteration over candidate placement points in the y -dimension is done inside the iteration over the x -dimension, and as a consequence of z -height not being a considered factor in this algorithm other than disallowing out-of-bounds placements, most boxes end up being placed against the left wall at $x = 0$. As demonstrated in Figure 4.2, this leads to a highly vertical, column-like packing structure. The limitations of this algorithm are discussed further in Section 6.1.

Pallet Results: **FFD** | Order 16 | Given Orders



Metrics Summary

-- Run Information -----		
Algorithm:	FFD	
Order ID:	16	
Boxes in order:	32	
Dataset:	Given Orders	
BnB Opt. Guarantee:	N/A	
BnB Candidate Limit:	N/A	
Nodes evaluated:	1,166	
-- Packing Metrics -----		
Max Z Height:	1399	mm
CoG Z-height:	628.92	mm
Packing Score:	0.26	
Order Fulfillment:	100.0	%
Volume Utilization:	23.59	%
Area Filled at z=0:	28.21	%

Figure 4.2: An example of FFD algorithm box placement.

4.3 Best Fit Decreasing

Building directly on the concept of FFD, and also first introduced by Johnson (7), the best fit decreasing algorithm was also adapted for the problem discussed in this work. In this algorithm, rather than committing to the first possible placement, every possible placement is simulated and assessed, and from there, the best placement is chosen. This leads to a considerable improvement in packing performance, and allows this algorithm to serve well as a solid naive algorithm baseline for more complex attempts at solving packing problems.

The algorithm works by first sorting the order to place by volume, and iterating through the sorted list. For each box, the algorithm generates all valid orientation dimension tuples, and initializes a best found score variable, filling it with the worst score possible based on the metric to assess. The algorithm is capable of assessing pallet states by any of the four built-in metrics (*max_z* (the default), *cog_z*, *packing_score*, and *volume_utilization*, see Section 3.3). As the former two are metrics to be minimized, the score is initialized as 100 mm above the pallet's upper boundary, guaranteeing that any subsequent score is lower. The latter two metrics being metrics to be maximized, their score is initialized as -1 , guaranteeing that any subsequent score is higher. Next, the pallet's current list of candidate points is loaded, and the algorithm iterates through every combination of orientation and candidate placement for the current box.

Enabled by the pallet environment's ability to both place and remove boxes, the algorithm calculates the pallet's score in the chosen metric for each possible virtual placement of the box. During the process of checking all of these placements, the score is compared to the best score found so far, and if the current score beats the best score, the placement currently being assessed is saved. At the end of the loop, the best placement for that particular box is used as the actual placement for the box, and the iteration process starts again, keeping count of every placement considered. An example of an order packed using this algorithm can be seen in Figure 4.3.

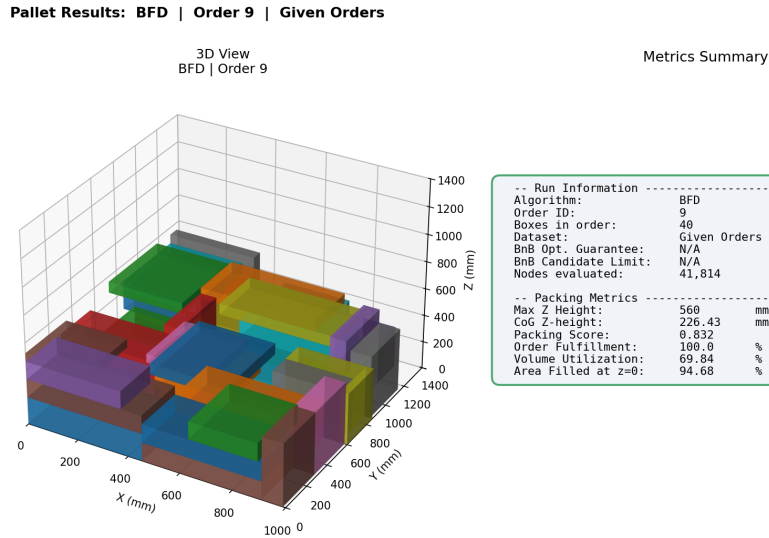


Figure 4.3: An example of BFD algorithm box placement.

4.4 Branch and Bound

4.4.1 Overview

At its core, the algorithm discussed in this section is a branch and bound algorithm as detailed in Section 2.4. This means that a search tree is created, with every node in the tree representing a (partial) pallet state of placed boxes, and every branch representing an additional potential

placement of another box. This means that every level traversed in the tree corresponds to the placement or removal of a single box. Utilizing the box removal capability of the pallet environment and the recursive nature of branch and bound algorithms, this algorithm is theoretically capable of assessing every possible solution in the full search space.

Searching the entirety of the possibility space is not the desired behavior, however. Given that the computational complexity of this problem exponentiates with every additional box to place, a full search would become intractable quickly. This is why this algorithm employs two methods of reducing the size of the search tree, namely bounding rules and branching filters. Bounding rules are conditions that are checked during the assessment of a node. If these conditions are (not) met, the node's entire potential subtree is immediately discarded from the search tree. Branching filters are conditions that are checked for each potential placement decision (branch). If these conditions are (not) met, only the single branch being considered is pruned. This algorithm uses a total of five different bounding rules and branching filters to achieve the performance goal of packing 40-box orders in tractable timeframes. While the algorithm's default mode of operation does not guarantee that an optimal solution to a problem instance will be found, it is capable of running in a way that preserves the guarantee of optimality. It does this by disabling the deduplication and top-X limiting filters, described below. Details about the impact of the differences in performance between these modes are discussed in Section 5.3. An example of an order packed using this algorithm can be seen in Figure 4.4.

4.4.2 Initialization and Warm-start

As is the case for most algorithms detailed in this chapter, the given order to place is first sorted by volume. After that, storage is initialized for the current sequence of $(dims, x, y)$ tuples containing box dimensions and placement locations, representing all required information to reconstruct a full pallet state, as well as storage for the best sequence found.

As the default bounding rule constrains the search space the most when a tight upper bound is already found, the algorithm uses a warm-start approach. This entails running the BFD algorithm on the order to place and measuring the maximum z -height of the resultant pallet state, and considering a score slightly worse than this score as the best score found. This ensures that the default bounding rule does not discard a state on par with the BFD algorithm, in case that the BFD algorithm has found the optimal solution already.

Finally, a constant-time lookup dictionary is generated containing the set of valid orientation dimension tuples per box in the order, followed by precalculation steps for the tall-low bounding rule and the symmetry breaking branching filter, detailed in their respective sections.

4.4.3 Recursive Structure

After initialization, the recursive function is defined. Its general structure cycles for each node, checking against the base case of all boxes having been placed, leading to termination and state reconstruction, followed by the bounding cases that lead to node removal, and the branching cases that prevent new branches from being spawned. If none of the previous cases apply, a new branch is spawned and the function recurses one level down in the search tree. After the recursive computation of the best sequence of placements has completed, the best found pallet state is reconstructed from the found placement sequence, and the tree statistics, consisting of trigger counts of each bounding rule and branching filter, the total amount of search tree nodes traversed, the best found state's z -height, the status of the optimality guarantee, and the value of the top-X limit used, are returned.

4.4.4 Bounding Rules

The most basic function of a branch and bound algorithm aimed at finding an optimal solution to a problem is the ability to reject suboptimal solutions. Named the default rule to signify this

fundamental methodology (sometimes referred to as the trivial rule or rule 1), this bounding rule is the first to enable the functioning of this algorithm. If the current pallet state being evaluated by the algorithm exceeds the previously best found score, the node is discarded and prevented from being considered, as a partial pallet state already evaluating as worse than the best full pallet state can never become the new best.

Another rule introduced as an evolution of the default rule, the so-named tall-low rule serves as an optimistic lookahead bound for packing height. Before the recursive algorithm is initiated, a dictionary is generated with, for each box in the order to pack, the dimensions of the tallest remaining box in the order. This enables the runtime functionality to check ahead for every partial pallet state, whether the tallest remaining box, placed at any extreme point candidate placement, in any orientation, exceeds the upper bound of the best score found so far. In the same logic as the default rule, if a box that will have to be placed in the future already cannot be placed without going over the best score found, the pallet will never be packed tighter.

In the development process for this algorithm, other bounding rules were conceptualized and implemented, but eventually removed due to lack of effectiveness. More on the (lack of) impact of these rules can be found in Section 5.3. The first of these was a volume-bound rule, whereby a dictionary was registered and populated by the minimum amount of box volume still to place for every level of the search tree, forming an absolute lower bound of z -height yet to be filled in the pallet, similar to Martello *et al.*'s L_0 bound (11), but applied partially for each level's set of remaining boxes (which is always the same across a level, as the sorting of an order is constant). The rule checked whether the box volume to place would exceed the z -height ceiling of the best completed packing yet, pruning the branch if so. As L_0 already showed, however, absolute lower bounds are of little use in packing problems, as evidenced by the fact that the volume-bound rule never pruned a single branch in any of its attempted iterations. Due to the small but measurable cost of its computation and its nonexistent effect on search tree size, the rule was eventually removed from the algorithm.

Another attempted bounding rule was conceived based on look-ahead functionality. The rule was intended to check if the subsequent box to place at the lowest level placement in its most favorable orientation would exceed the z -height ceiling of the best completed packing yet, pruning the branch if so. As is known now, the lowest level placement of virtually every partial or full packing exists at $z = 0$, due to the fact that it is nearly impossible to achieve a perfectly closed packing of the floor of the pallet. Therefore the rule would only consider boxes placed on the floor of the pallet, and thus never fire. For the same reasons as the volume-bound rule, this rule was also deprecated.

4.4.5 Branching Filters

The first branching filter, termed deduplication, is an aggressive rule designed to trade the guarantee of the algorithm finding the optimal solution for an $11.25\times$ reduction in node traversal, effectively shrinking the search space to 8.89% of its original size (see Section 5.2). This branching filter, when enabled through the relinquishment of the optimality guarantee, keeps track of a set of so-named profile keys, which consist of a (*dimensions, z -height*) tuple. A profile key effectively constitutes the combination of a box orientation and the height at which it is placed. Whenever a box placement decision passes all checks and the branch is allowed to become a node in the search tree, its profile key is recorded. If the same box, placed at a different extreme point would end up being placed at the same height, the branch is filtered by this rule. Note that the placement location of the box is not a factor in the decision of whether or not to filter a branch. This lack of exactness is the reason why the guarantee of optimality is sacrificed by this rule, as the possibility that a branch discarded by this rule could lead to the optimal packing remains.

The second branching filter, named the symmetry breaking filter, was originally envisioned as an optimality guarantee-preserving version of the deduplication filter. It quickly proved itself as a

strong method of reducing search space size that is not made obsolete by deduplication filtering. Based on the idea of finding redundant states in the search space discussed by Zhang (2021)(15), the symmetry breaking filter finds potential states of identically sized boxes in interchanged places, and blocks the branching when they occur. Empowered by the fact that there are only ten types of boxes, two of which share identical dimensions (box types 8 and 10 are both $375\text{ mm} \times 260\text{ mm} \times 130\text{ mm}$), every order of 10 boxes or more is guaranteed to have at least two boxes of the same dimensions, thus allowing the symmetry breaking filter to trigger in almost every situation. Functionally, the filter operates in three different phases of the algorithm. Prior to entering the recursive loop, a list of orientation-agnostic dimension tuples for every box in the order to be placed is registered. This is achieved by sorting the dimension tuples in ascending order, and is done so that the algorithm can detect identical dimensions regardless of box ID. Inside the recursive loop, if no previous bounding rule has pruned the node, a check is performed to see if the orientation-agnostic dimension tuple of the next box to place matches that of the previously placed box. In other words, the box is checked for being dimensionally identical to the previous box. If this is the case, the combination of the orientation tuple and placement location of the previous box is registered as a symmetry key. During the branching process, in which every potential placement of the current box is evaluated, the orientation tuple and placement location combination of every potential placement of the current box is checked against a registered symmetry key, if any. If the potential placement configuration is lexicographically smaller than the symmetry key, it indicates an interchangeable placement of identically sized boxes, and thus forms a redundant symmetric state. If this is the case, the branch is filtered, ensuring only unique representative branches for each symmetric possibility are explored.

The final, and arguably most powerful, branching filter is one that caps the growth of the search tree by a fixed amount per node. Named the top- X limiting filter, it is based on the theory of beam search, but adapted in a small but meaningful way with the intention to balance the tradeoff between loss of packing performance and reduction of computational complexity a bit more in favor of preservation of packing performance. Top- X limiting works by restricting the number of branches each node is allowed to spawn, rather than restricting the total amount of spawned nodes per layer of the search tree. In this way, it is much more likely that the optimal path down the search tree will be found, while still dramatically lowering the amount of nodes that are to be evaluated, even though the exponential nature of the tree is not remedied. In Section 5.4, different values of this limit are tested for their positive and negative impact on computational complexity and packing performance, respectively. Despite its structural simplicity, this branching filter has a profound impact on search tree size reduction. Enabled by the fact that in order to be able to branch, all potential valid branches must be identified, the algorithm can simply register an additional value during the identification process. Including a comparison tuple consisting of the top z -value reached by the candidate placement, as well as a rough score of how far away from the origin in the x and y -dimension the box is placed, calculated by simply adding placement x and y values together, into the candidate placement encoding, the branching filter can simply lexicographically sort every placement candidate's comparison tuples, and select only the top X to allow branching. As this branching filter cannot guarantee that it leaves the optimal path open for exploration, it is only enabled if the guarantee of optimality is relinquished.

Similarly to the development of the aforementioned bounding cases, two failed attempts at branching filters were also conceptualized and implemented before being deprecated. In Section 5.2, the implementation and removal of these filters are detailed. The first of these, a branching filter that could potentially realize a significant reduction in search tree size, was theorized and implemented as the heightmap recognition rule. Its function was to keep and update a registry of previously seen heightmap hashes, and block branching if a box placement would create an identical heightmap. It was envisioned as a method to catch duplicate states that would not be recognized by either the deduplication or symmetry breaking filters, but its

computational impact was much too great in relation to its impact on search tree size, and it was immediately abandoned.

Another branching filter aimed at securing proper placement of boxes was created. Named the two-surface filter and equipped with the functionality to block any branch that would not place a box that rested against at least two surfaces, be it pallet boundaries or the sides of previously placed boxes, it was intended to enforce leftward and backward packing to leave as much usable area at every height, while maintaining the possibility of optimal packing, as proven by Martello *et al.* (11). While this filter did report as being triggered often, it did not result in meaningful gains in search tree reduction or computation time. The suspected reason for this is that it serves only as a redundant computation of the extreme point candidate placement selection, which is already applied in the algorithm.

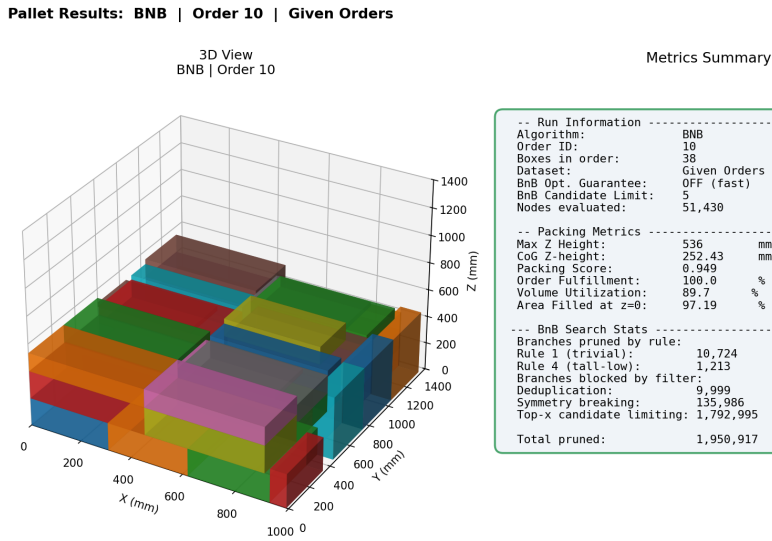


Figure 4.4: An example of BnB algorithm box placement.

5. Experiments and Results

5.1 Random Algorithm Order Fulfillment Test

From the perspective of an algorithm, box placement only consists of an attempt in the form of a box dimension tuple and two-dimensional placement coordinates, and seeing as how the pallet environment decides whether or not such an attempt results in the box actually being placed (see Section 3.2), it quickly becomes clear that an algorithm that attempts to place boxes at random coordinates is unlikely to place every box in an order on the first try. A test was designed and conducted to investigate the relationship between allowed attempts and order fulfillment.

The testing function takes as input the start and end to a range of order numbers, the order set to read from, the number of unique orders to sample, the upper limit on the number of maximum-attempts values to test, the step size for maximum attempt values up to the maximum, and a toggle for saving the results figure. When it is run, the specified number of orders are sampled evenly from the input set, the range of maximum attempts values is generated, and the algorithm is run on the sampled orders for each value in the range of maximum attempts values. Running the test on an evenly spaced sampled subset of 500 type II test orders out of the full 1000-3999 range to represent all box counts, and testing up to a peak maximum attempts value of 10000

and a step size of 100 yields the results seen in Figure 5.1a.

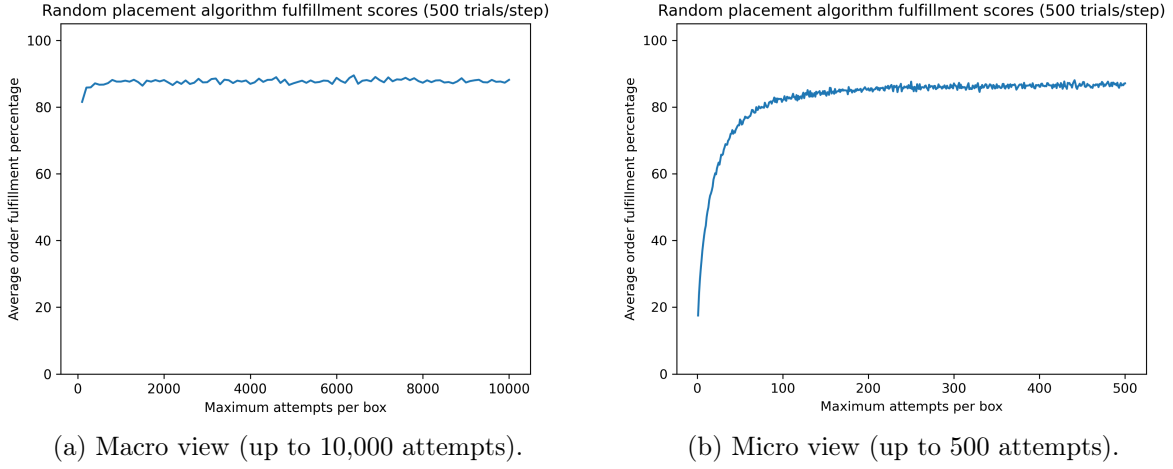


Figure 5.1: The average order fulfillment percentage per order per pallet of the random algorithm as a function of maximum allowed placement attempts per box.

Interestingly, increasing the maximum attempts value over the 1,000 mark did not make any meaningful impact on the order fulfillment percentage of the random algorithm, hovering around the 85 to 90% range. This shows that, as can be seen in Figure 4.1, the random algorithm may simply put itself in a position where no more valid box placements can be found before fulfilling the order. At the smallest maximum attempt values, however, the small uptick at these values reflects the initially expected results of this test: an increase of attempts slightly elevates the number of boxes the algorithm is generally able to place. This can be seen in more detail in Figure 5.1b. In conclusion, the random algorithm may be executed in its most efficient way with a maximum attempts value of only 250.

5.2 Branch and Bound Development

As detailed in Section 4.4, a number of different features and capabilities were added and removed during the development of the branch and bound box placing algorithm. The additions and removals of bounding rules and branching filters all had an effect on the search tree size, node traversal speed, and, following from these two factors, algorithm completion time. To quantify the effect of these changes, the algorithm was run on test order 15, a 15-box type I test order, and a number of metrics were recorded after every addition, removal, and change of the branch and bound algorithm logic, including completion time, search tree size, and node traversal speed. The results of these runs can be seen in Table 5.1.

As can be seen in Table 5.1, the baseline branch and bound algorithm, consisting only of its most base functionality and the default bounding rule, took more than 17 minutes to place a 15-box order. Adding the BFD algorithm warm-start functionality introduced a slight decrease in processing time, and while it did little to reduce the size of the search tree (as it was observed that the algorithm converged on its final score relatively quickly after initialization), it was deemed not to be a burden on processing speed, being performed outside of the recursive loop, and has remained part of the algorithm's functionality ever since. The volume bounding rule and look-ahead rule were added next. As mentioned in Section 4.4.5, these rules proved ineffective at reducing the search tree size by even a single node. The introduction of the tall-low rule provided a significant improvement, however. It reduced the search space by more than half its size, and even at the cost of reducing the node traversal speed by roughly 36%, the completion time was successfully reduced by several minutes. After this, the final iteration of the volume

Table 5.1: Performance metrics during algorithm development on Test Order 15 (15 boxes).

Algorithm Modification	Completion Time	Nodes Evaluated	Avg. Nodes/s
Base algorithm	17m 03s	4,354,054	4,462
BFD warm-start	16m 45s	4,349,175	4,374
Volume rule addition	17m 38s	4,349,175	4,243
Volume rule refinement	16m 49s	4,349,175	4,360
Look-ahead rule addition	17m 02s	4,349,175	4,327
Tall-low rule addition	13m 17s	2,122,024	2,745
Volume rule final form	13m 55s	2,122,024	2,576
Look-ahead rule deprecation	13m 43s	2,122,024	2,584
Deduplication filter addition	0m 29s	188,559	1,714
Symmetry breaking filter addition	0m 03s	4,329	1,246

bounding rule was implemented, as it was still considered capable of effectuating search tree reduction pending further improvements, while the look-ahead rule was deprecated as it was deemed fundamentally ineffective. When the first two branching filters (see Section 4.4.5) were introduced, the first substantial gains in search tree reduction were realized. While the node traversal speed was reduced by a third on introduction of the deduplication filter, the search tree was reduced by 91%, enabling the first sub-minute run of the algorithm. When the symmetry breaking filter was introduced, the solving of 15-box problem instances became computationally trivial with the total reduction in search tree size reaching ~99.9%. At this point, further tests were performed on test order 25, a 25-box type I test order. Results of further iteration can be seen in Table 5.2. Note that the first row of this table represents the same algorithm as the last row of Table 5.1, and that only a more difficult problem instance was introduced between these two rows.

Table 5.2: Performance metrics during BnB algorithm development on Test Order 25 (25 boxes).

Algorithm Modification	Completion Time	Nodes Evaluated	Avg. Nodes/s
Problem difficulty increase	24m 48s	1,766,249	948
Volume rule deprecation	23m 31s	1,766,249	971
Heightmap rec. filter addition	61m 55s	1,756,995	424
Heightmap rec. filter deprecation	23m 36s	1,766,249	986
Two-surface filter addition	23m 54s	1,728,994	905
Two-surface filter deprecation	23m 37s	1,766,249	986
Top-X limiting filter addition	5m 35s	79,388	303

In Table 5.2, it is clearly shown that the complexity of the problem discussed in this work grows immensely as more boxes are introduced to the problem instance. Keeping in mind the existing ~99.9% reduction in search tree size from the algorithm modifications detailed in Table 5.1, it can be estimated that the search tree size of even a 25-box problem instance for the base version of the algorithm can reach 1.7 billion nodes, which would make the base algorithm take an estimated 110 hours to complete. Considering that solving the 40-box orders of the Centraal Boekhuis order set would not be tractable with the algorithm as it was, the algorithm speed and efficiency required improvement. After further experimentation, the volume bounding rule was deprecated as it became clear that it could not prune the search tree in any form. Following this, two more branching filters were conceptualized (see Section 4.4.5). First, the heightmap recognition branching filter was introduced to prevent duplicate heightmaps from being computed

redundantly. Upon realizing that this filter’s functionality depended on the hashing of a 1400×1000 heightmap for each of the 1.77 million nodes, accounting for multiple terabytes of memory writes and garbage collection and an impairment of node traversal speed of more than half, the filter was deprecated. The penultimate modifications to the algorithm consisted of the addition and near-immediate deprecation of the two-surface filter, which had a negligible impact on both search tree size and completion times. Finally, however, the top-X branching filter was implemented, which, when set to limit to 5 branches per node, reduced the search tree to $\sim 4.5\%$ of its pre-implementation size. With this adjustable branching filter in place and this level of performance achieved, the algorithm was deemed fit for purpose.

5.3 Optimality Guarantee Test

The creation of the deduplication and top-X limiting branching filters, along with the ability to toggle the relinquishment of the guarantee of finding an optimal solution to dramatically decrease search tree size, necessitates a thorough investigation into the effect of this trade-off. For this investigation, the branch and bound algorithm was run on all type II test orders with box amounts ranging from 10 to 25 (out of the full range of 10 to 39 boxes), with and without the guarantee of optimality, and a number of metrics were recorded about these runs, including the packing performance and search tree size. In Figure 5.2, a selection of the results of this test are shown.

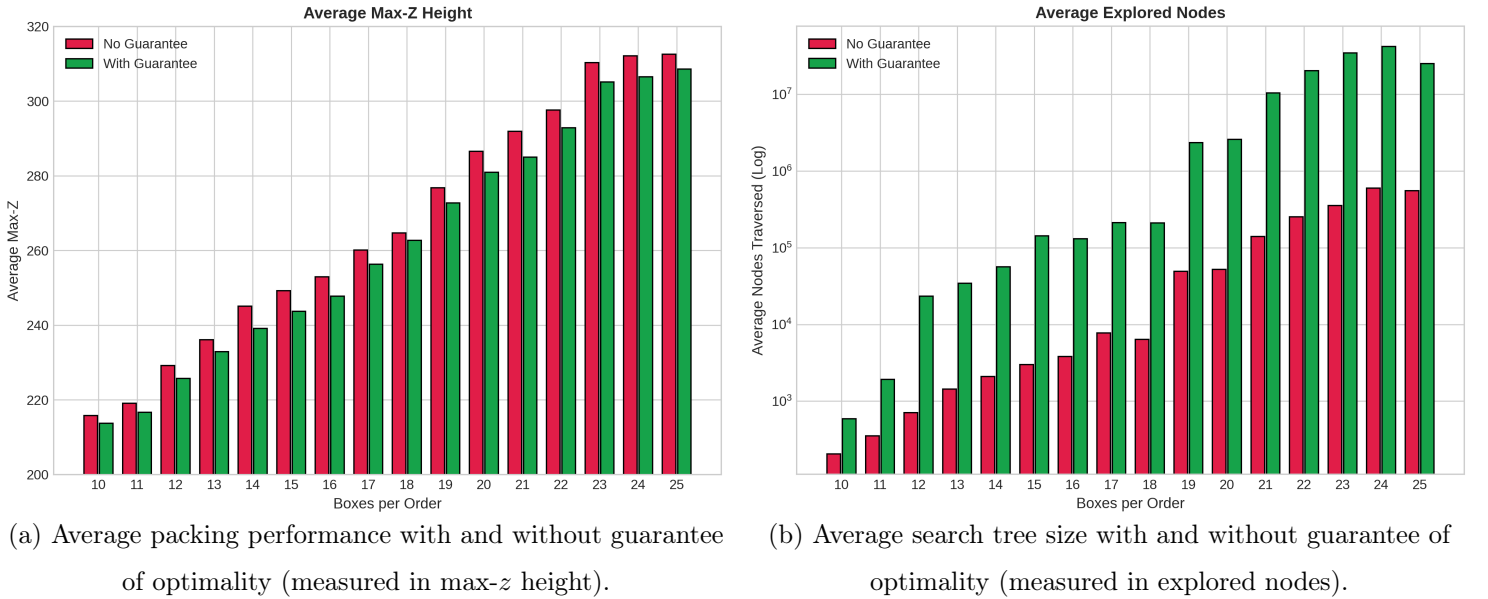
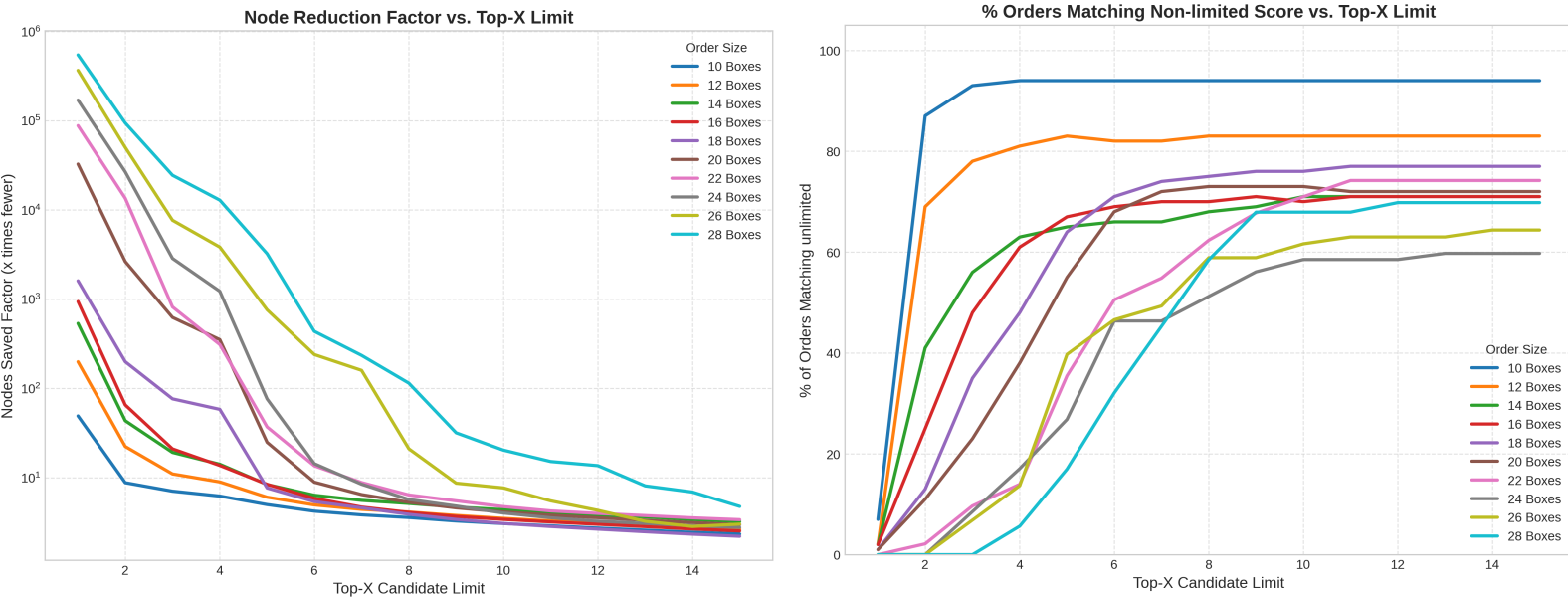


Figure 5.2: The branch and bound performance averages with and without optimality guarantee versus problem instance size.

As can be seen in Figure 5.2, most results of this test are in line with intuitive expectations. The average max-z height, as well as the search tree size, increases on average as more boxes are to be placed, irrespective of the guarantee of optimality. Furthermore, the packing performance of the algorithm with the guarantee intact performs better in terms of packing performance, on average, for every size of problem instance. What is interesting, however, is the fact that while maintaining the guarantee increases the search tree size, on average, by a factor of 44, the average increase in packing performance achieved by maintaining it only amounts to a 2% lower max-z height achieved. The results of this test strongly indicate that relinquishment of the optimality guarantee is warranted in real-world packing scenarios, as the drastic search tree reduction far outweighs the relative insignificance of packing performance that is sacrificed.

5.4 Top-X Limit Test

With the implementation of the top-X limiting branching filter (see Section 4.4.5), the branch and bound algorithm is equipped with a strong, adjustable branching filter that can flexibly balance the trade-off between aggressive search tree size reduction and probability of finding optimal solutions. Finding the optimal balance for this filter requires the testing of different limit values, however. A test was conducted wherein the branch and bound algorithm was run on all type II test orders containing up to 28 boxes, with top-X limit values of 1 through 15, as well as with top-X limiting disabled. From these runs, data was collected including solution time, search tree size, and packing performance. Processing these data revealed the relationships between top-X limit values and search tree size (see Figure 5.3a) and between top-X limit and performance matching non-limited runs (see Figure 5.3b)



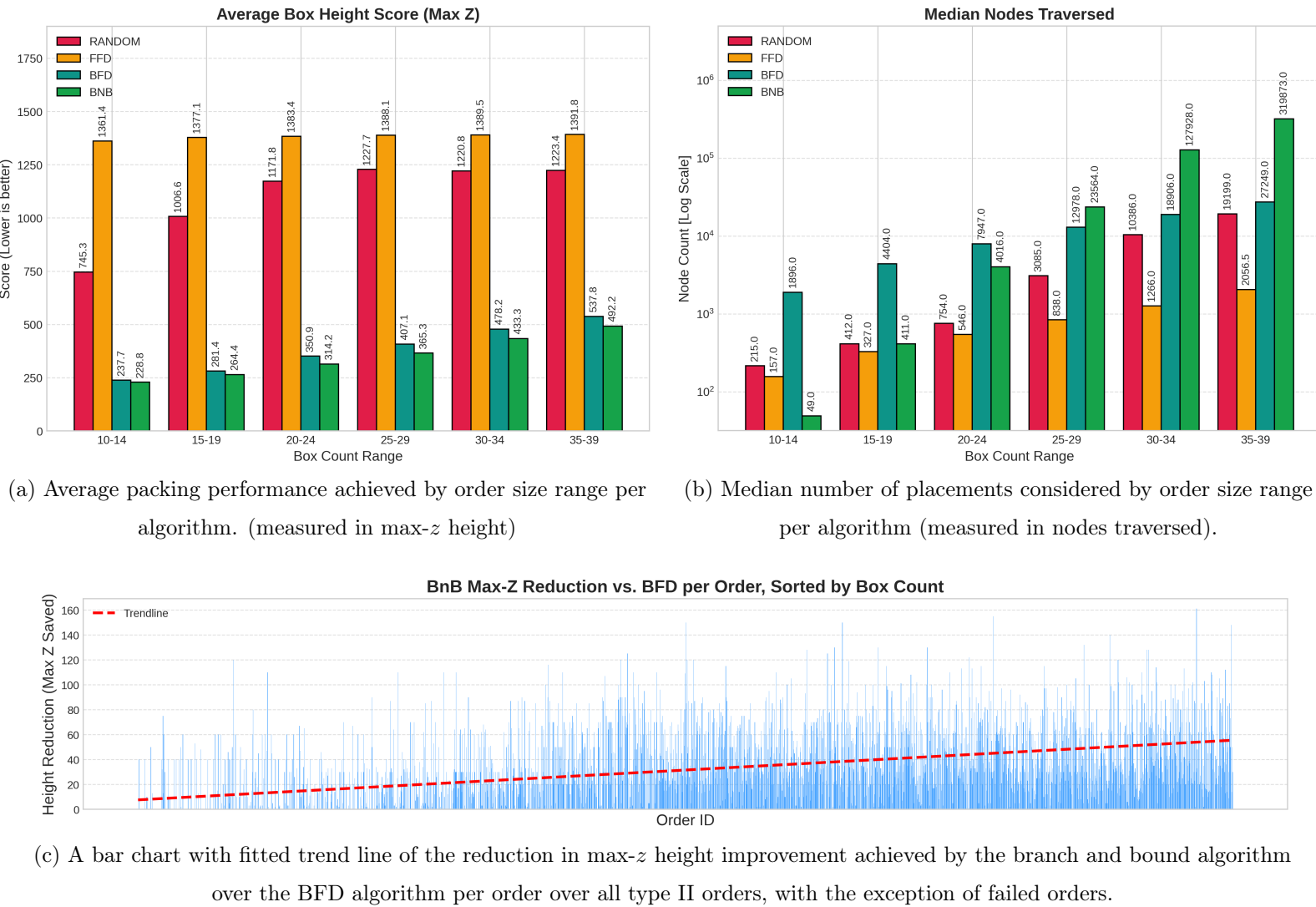
(a) Search tree size reduction factor versus top-X limit. Each line represents up to 100 problem instances of equal size. (b) Percentage of order solutions per box amount that matches non-limited packing performance versus top-X limit value.

Figure 5.3: The effects of different top-X limit values compared to non-limited runs.

As can be seen in Figure 5.3a, the number of boxes in the problem instance has significant effect on the efficacy of different top-X limit values. While a search tree size reduction factor of over 50,000 can be achieved with the strictest possible top-X limit in the largest tested problem instances, this effect drops off sharply as problem instance size decreases, with 10-box orders benefiting from a relatively small reduction factor of approximately 15. It is clear, however, that the search tree size reduction scales strongly with top-X limit values, no matter the problem instance size. In Figure 5.3b is shown the percentage of run solutions that reach performance parity to non-limited runs per box amount. It can be seen that as the problem instance size increases, the average top-X limit required to maintain parity also increases somewhat linearly, and that even the highest tested values of top-X limits could not reach full parity with non-limited runs. Based on the fact that a top-X limit value of 5 is proven to lead to much more significant reductions in search tree sizes compared to limit values of 10 and up, yet maintains performance parity in about 50% of the cases that a limit value of 10 achieves, the default limiting value of the top-X branching filter was set at 5.

5.5 Algorithm Comparison

With primary contribution of this work, the branch and bound algorithm, completed, and with the performance of its settings analyzed, a comprehensive test of its capability and potential drawbacks became necessary, particularly in comparison to the three baseline algorithms discussed in Chapter 4. This test includes running each algorithm on an order or set of orders (in the case of the results below, all type II test orders), and, similarly to the two previous tests, recording performance metrics including node traversal, completion time, packing performance, and all other metrics of packing performance and search tree pruning. While the vast majority of orders completed successfully, 359 orders were not successfully solved by the branch and bound algorithm. These failures are discussed in Section 6.4. After the test, the results are binned into six problem difficulty classes based on box amount per order. A selection of results of this test can be seen in Figure 5.4.



algorithms almost always stack boxes close to the upper bound of the pallet, making the BFD algorithm the only serious baseline to compare the branch and bound algorithm to. Compared to the BFD algorithm, the average packing performance improvement the branch and bound algorithm achieves is small, but noticeably increases with increasing problem instance sizes, with an average of approximately 9 mm of z -height saved in the 10 to 14 boxes per order range increasing to an average of approximately 45 mm saved in the 35 to 39 boxes per order range. In Figure 5.4c, the max- z improvement amount that the branch and bound algorithm achieved over the BFD algorithm is shown for every order in the testing set, with the exception of the failed orders. A fitted trend line is overlaid on this graph, showing that as the order ID, and thus the box amount per order, increases, the average performance improvement of the branch and bound algorithm also increases. Across all tests, 735 instances of a max- z improvement of 50 mm or more achieved by the algorithm over BFD were recorded, proving the efficacy and utility of the branch and bound algorithm proposed in this work.

The branch and bound algorithm consistently performs better in terms of packing efficiency. This performance comes at the cost of computational complexity, however. In Figure 5.4b, it is clearly visible that the branch and bound search tree size increases exponentially with increasing order size, and, at the largest problem sizes, exceeds the node traversal count of BFD by an order of magnitude. While it may seem that the branch and bound algorithm is less computationally demanding than the BFD algorithm at smaller order sizes due to having a smaller number of nodes to traverse, the significantly higher algorithmic complexity of the branch and bound algorithm leads to longer median completion times in all problem size classes (see Figure A.3, top-right graph). The significant computational cost of the branch and bound algorithm, though worthwhile in packing performance, does not invalidate the use of heuristic algorithms in all cases, especially in the largest problem instance size classes, and serves as a potent reminder that performance always comes at a cost.

6. Discussion

In this chapter, some of the weaknesses of the algorithms described in this work, as well as suggestions for future work to improve upon, are discussed.

6.1 FFD Inapplicability

While there is a case to be made for the implementation of greedy heuristic algorithms in any work discussing complex problems, it is clear that Johnson’s (7) FFD algorithm, originally made for the classical 3D bin packing problem, is not readily applicable to the problem type discussed in this work. In the transposition of the concept of ‘first fit’ from the 3D-BPP to the 3D-RODP, a critical aspect of value is lost, as in the transposed version of the algorithm, the first valid placement of a box in a single pallet consistently translates to the left-back-bottom corner, in that specific order of importance. With height being the lowest-priority factor, it invariably results in stacking a tower of boxes in the left-back corner of the pallet from the floor to the upper bound of the pallet. If this work is to be revisited, a revised version of this algorithm should be considered that faithfully adheres to the first-fit philosophy, but takes care not to naively place boxes in a stack in the pallet’s corner.

6.2 Top-X Limiting Weakness

In the creation of the top-X branching filter (see Section 4.4.5), an effort was made to improve upon the strengths of beam search while mitigating some of its weaknesses. In other words,

the filter should maintain the capability of drastically reducing the size of the search tree, while being less susceptible to purely focusing the computational effort on local minima that could lead to missing the optimal solution. To accomplish these goals, the filter was designed in such a way as to limit branching on a local, per-node basis, rather than on a global basis. While this in theory allows every branch to be followed to its optimal conclusion, it loses a fundamental strength of beam search in the process. In allowing every node in the tree to branch, even if the number of branches is limited, the width of the tree grows exponentially with depth, whereas beam search restricts the tree growth to a linear pattern. Due to this weakness, order completion times reached 129 hours in the worst recorded case, which is an unacceptable amount of time. While this instance was an extreme outlier, as only the approximately 400 most computationally demanding solutions required more than an hour of processing time, it shows that combinatorial explosion is not ruled out by this method of filtering branches.

6.3 Physics Limitations

In this work, minimal consideration was made for real-world physics. While an accommodation was made in the form of the 70% supported area requirement for box placement validation, the state of the art of bin packing science has progressed far beyond rigid-body placement checks that only happen at the time of box placement. An example of this is the work of Wang *et al.* (12), wherein a physics simulation was implemented that included friction, gravity, and stack oscillation and collapse, that ran continuously as the bin was packed. The critique that Wang *et al.* express about many studies approaching bin packing problems from purely mathematical perspectives applies fully to this work. The real-world applications of efficient bin packing are numerous and valuable (see Chapter 1), but these applications are extremely limited if the constraints of the real world are not applied to the research. Future work should incorporate more real-world physics to investigate the applicability of the algorithm(s) proposed in this work in realistic scenarios.

6.4 Branch and Bound Failures

After the extensive tests done on the branch and bound algorithm, it has come to light that this algorithm suffers from a flaw in its implementation. Due to the interaction between the BFD warm-start and the branching filters that sacrifice the guarantee of optimality (see Section 4.4.5), there exist situations where the search tree is pruned in such a way that a box placement sequence that leads to an equal or lower max- z value compared to the BFD warm-start cannot be found by the algorithm. If this is the case, the algorithm fails and leaves the pallet unpacked. With default settings (no optimality guarantee, and a top-X limit of 5), the algorithm failed to find a sequence for 359 of the tested type II test orders, and 2 Centraal Boekhuis orders. In future versions of this algorithm, a fallback mechanism could be implemented that, for example, uses the BFD sequence as a backup in case the search tree does not provide a sequence to outperform or match the BFD algorithm.

7. Conclusion

In this work, the three-dimensional rectangular open dimension problem was modeled as a custom Python environment supporting three classical placement algorithms and one custom branch and bound algorithm synthesizing classical and modern methods of search tree pruning and optimization. These algorithms were leveraged to test their performance against one another utilizing a small real-world order dataset from the Centraal Boekhuis company, as well as a large

set of generated test orders of similar sizes.

The custom branch and bound algorithm consistently outperformed every other tested algorithm in packing performance, and while the computational complexity of this problem could not be fully controlled due to the lack of elimination of exponential search tree growth, the implementation of the custom top-X branching filter has proven to contribute significant value in improving upon the BFD baseline of packing performance in every class of order difficulty.

Finally, while this work has shortcomings pertaining to real-world constraints that were not represented, every improvement in the field of algorithmic solutions to three-dimensional packing problems can and will contribute to ongoing improvements in world logistics, saving labor, pollution, costs, and fuel.

References

- [1] IGNACIO ARAYA, MAURICIO MOYANO, AND CRISTOBAL SANCHEZ. **A beam search algorithm for the biobjective container loading problem.** *European Journal of Operational Research*, **286**(2):417–431, 2020. 4
- [2] HENRIK I CHRISTENSEN, ARINDAM KHAN, SEBASTIAN POKUTTA, AND PRASAD TETALI. **Multidimensional bin packing and other related problems: A survey.** *Computer Science Review*, **24**:34, 2016. 1
- [3] EDWARD G COFFMAN JR, MICHAEL R GAREY, AND DAVID S JOHNSON. **Approximation algorithms for bin-packing—an updated survey.** In *Algorithm design for computer system design*, pages 49–106. Springer, 1984. 1
- [4] TEODOR GABRIEL CRAINIC, GUIDO PERBOLI, AND ROBERTO TADEI. **Extreme point-based heuristics for three-dimensional bin packing.** *Inform Journal on computing*, **20**(3):368–384, 2008. 3, 4, 7
- [5] HARALD DYCKHOFF. **A typology of cutting and packing problems.** *European journal of operational research*, **44**(2):145–159, 1990. 2
- [6] M R GAREY AND D S" JOHNSON. *Computers and intractability: A guide to the theory of NP-completeness*". W. H. Freeman and Company", 1979. 1
- [7] DAVID S JOHNSON. *Near-optimal bin packing algorithms*. PhD thesis, Massachusetts Institute of Technology, 1973. 3, 8, 9, 10, 20
- [8] CARLOS LAMAS FERNANDEZ. *From archaeology to 3D printing: packing problems in the three dimensions*. PhD thesis, University of Southampton, 2018. 3
- [9] AH LAND. **An Automatic Method For Solving Discrete Programming Problems.** *Econometrica*, **28**(3):497–520, 1960. 3
- [10] BRUCE T LOWERRE. *The harpy speech recognition system*. Carnegie Mellon University, 1976. 4
- [11] SILVANO MARTELLO, DAVID PISINGER, AND DANIELE VIGO. **The three-dimensional bin packing problem.** *Operations research*, **48**(2):256–267, 2000. 3, 4, 12, 14
- [12] ZHOUFENG WANG, HANG ZHAO, JUZHAN XU, SHISHUN ZHANG, RUIZHEN HU, CHENYANG ZHU, ZECUI ZENG, WEIYAN ZHU, ZEYU XIONG, HAIBIN YU, ET AL. **RoboBPP: Benchmarking robotic online bin packing with physics-based simulation.** *arXiv preprint arXiv:2512.04415*, 2025. 21
- [13] GERHARD WÄSCHER, HEIKE HAUSSNER, AND HOLGER SCHUMANN. **An improved typology of cutting and packing problems.** *European journal of operational research*, **183**(3):1109–1130, 2007. 2
- [14] RICCARDO ZANCHETTA. *3D Bin Packing: a new Heuristic approach for real case scenario*. PhD thesis, Politecnico di Torino, 2025. 3
- [15] XIANGYI ZHANG. *Exact Algorithms for vehicle routing problems with two-dimensional loading constraints*. Ecole Polytechnique, Montreal (Canada), 2021. 5, 13

- [16] HANG ZHAO, QIJIN SHE, CHENYANG ZHU, YIN YANG, AND KAI XU. **Online 3D bin packing with constrained deep reinforcement learning**. In *Proceedings of the AAAI Conference on Artificial Intelligence*, **35**, pages 741–749, 2021. 1

A. Appendix

A.1 Source Code

The code written for this work, along with the datasets given and generated for this work, all generated results and full commit history can be found on GitHub at the following URL:
<https://github.com/xXxCodeSlayerxXx/thesis>

A.2 Reproducibility

With the exception of the explicitly random algorithms and measures of computation speeds, all results in this work are fully reproducible from source code and provided data. The Conda environment used can be replicated with the `environment.yml` file below. The file can also be found in the root directory of the GitHub repository.

```
name: thesis
channels:
  - https://conda.anaconda.org/conda-forge
dependencies:
  - pandas
  - numpy
  - tqdm
  - ipykernel
  - matplotlib
  - ipywidgets
  - pip
prefix: /home/william/.conda/envs/thesis
```

A.3 Full Results Figures

Below can be found the full, uncompacted result sets of the experiments discussed in Chapter 5. All other results, including results of tests not discussed and the full pallet and data renders of all Centraal Boekhuis orders, can be found in the GitHub repository in the `./results/` directory.

A.3 Full Results Figures

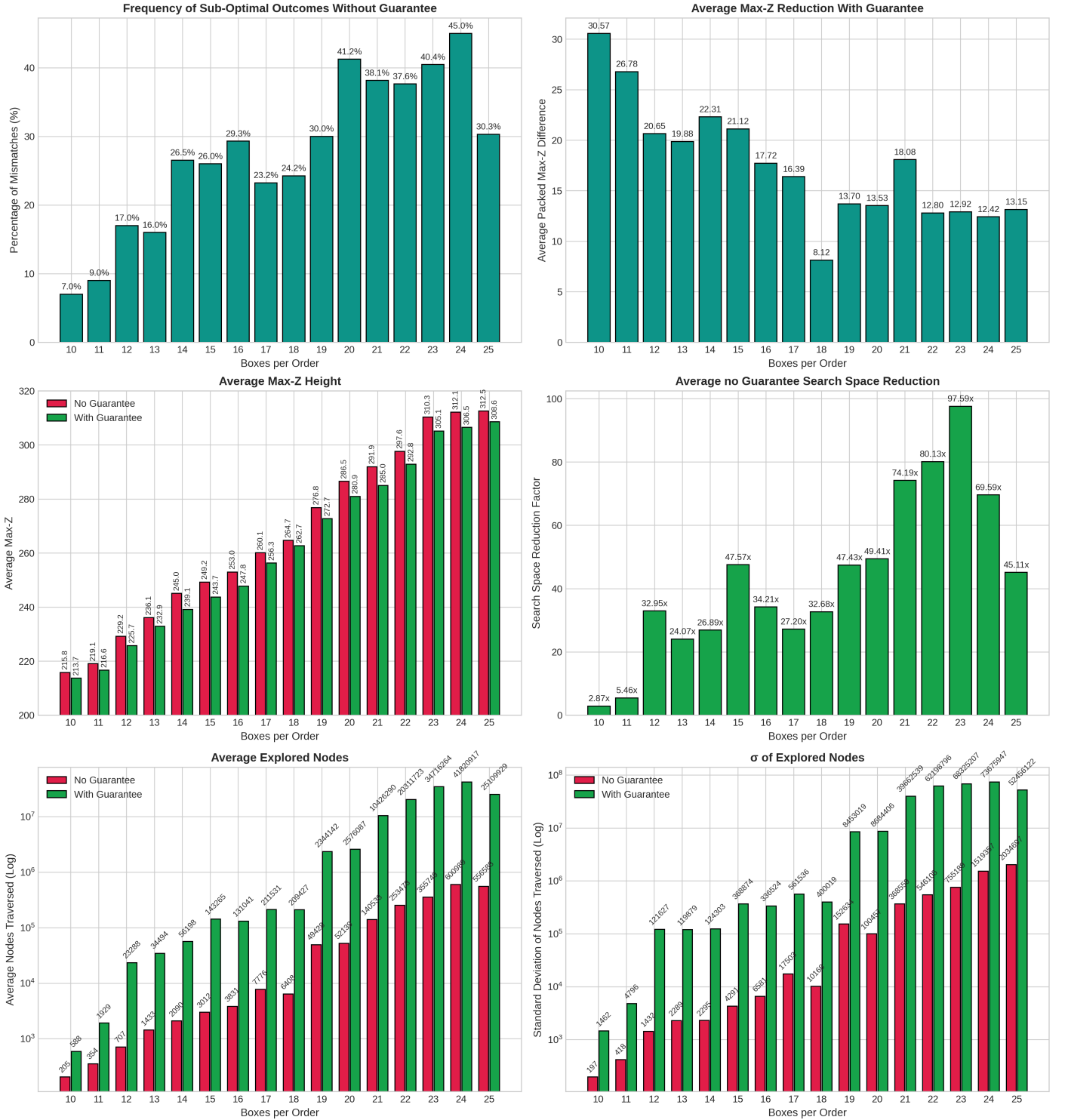


Figure A.1: The unreduced results of the algorithm comparison test.

A.3 Full Results Figures

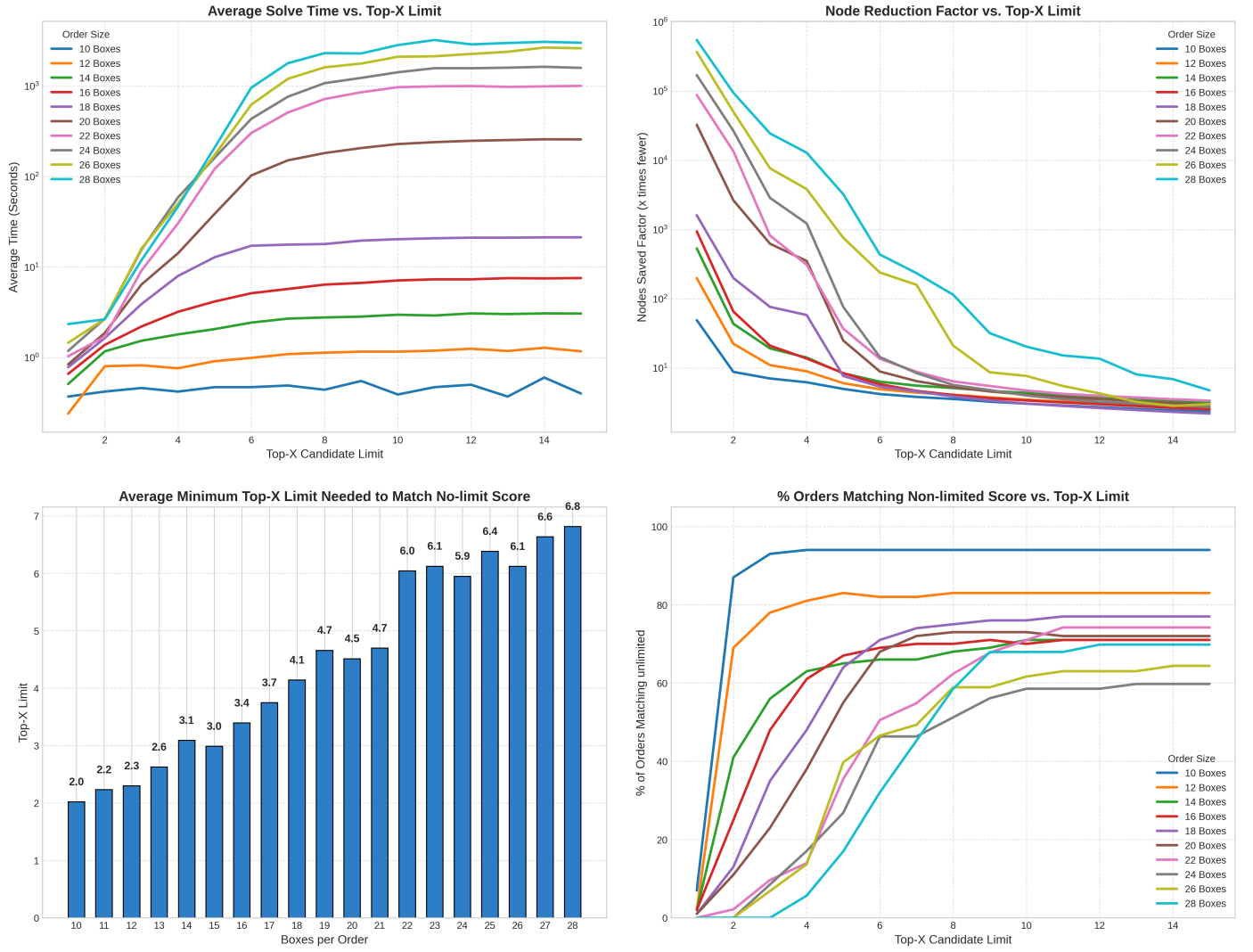


Figure A.2: The unreduced results of the top-X limit test.

A.3 Full Results Figures

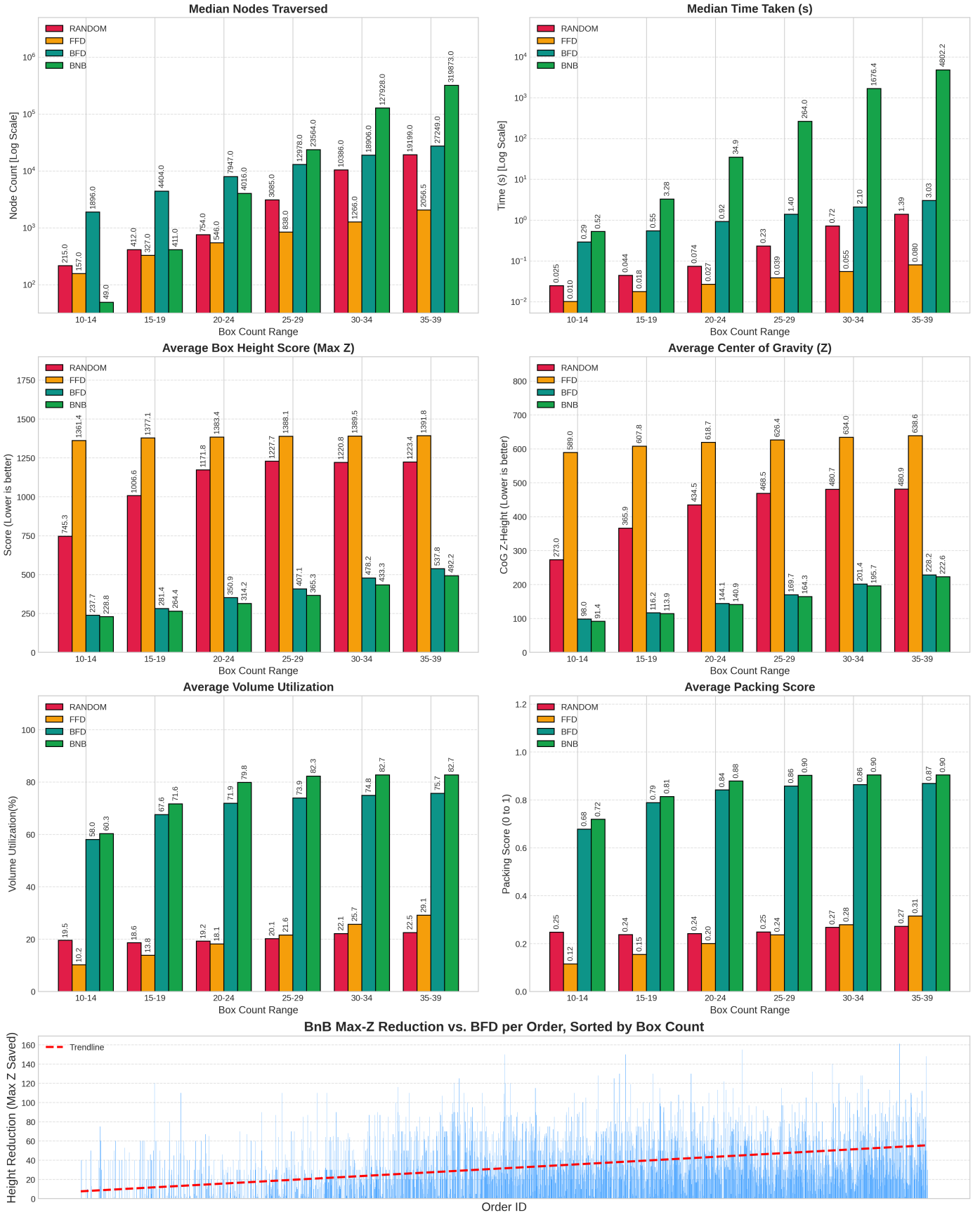


Figure A.3: The unreduced results of the algorithm comparison test.

A.3 Full Results Figures

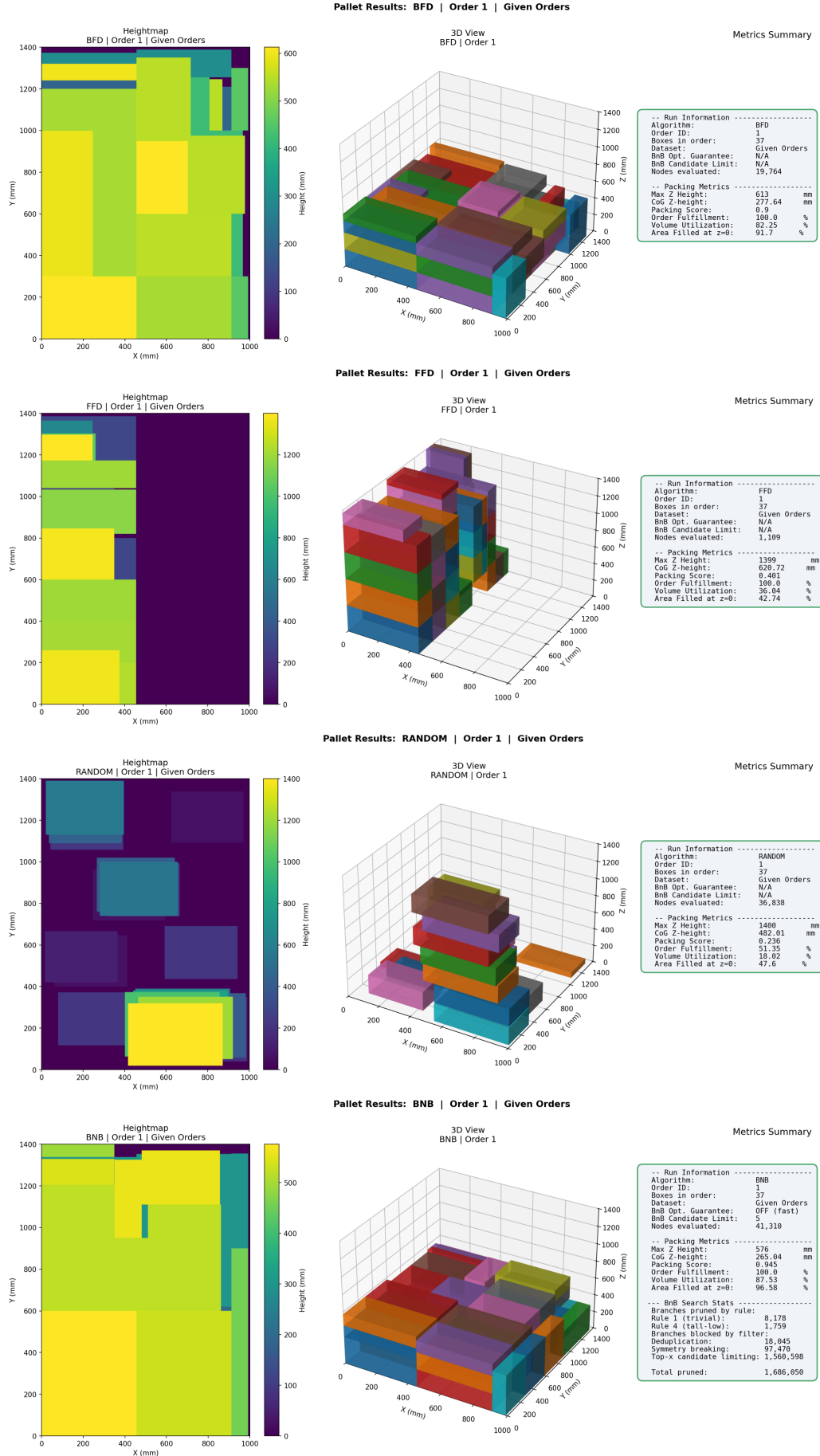


Figure A.4: The packing of Centraal Boekhuis order 1 by all algorithms, including heightmap.